

PPS Single Module Development Guide (PPS 單一波形圖開發指南)

本指南以 `pps-single` 為範本，說明如何開發一個具備「高頻波形數據視覺化」、「多維度數據縮放」以及「自動輪詢機制」的圖表模組。

1. 架構概觀 (Architecture Overview)

此類模組通常包含以下四個主要部分：

1. **Backend Route**: 定義頁面與 API 路由。
2. **Backend Controller**: 處理數據查詢與 View 渲染。
3. **Frontend View (EJS)**: 負責 HTML 結構、i18n 注入與依賴引入。
4. **Frontend Logic (JS)**: 負責 ECharts 渲染、數據歸一化/縮放、自動輪詢 (Polling)。

2. 後端開發 (Backend Development)

Step 1: 定義路由 (Route)

在 `routes/` 目錄下定義兩個端點：一個用於回傳 HTML 頁面，一個用於回傳 JSON 數據。

```
// routes/yourModule.js
router.get("/page", controller.getPage);    // 頁面進入點
router.get("/data", controller.getData);    // 數據 API
```

Step 2: 實作控制器 (Controller)

在 `controllers/` 目錄下實作邏輯。

- **getPage**: 渲染 EJS 模板，並傳入初始參數 (如 `type`)。
- **getData**: 接收前端參數 (`type, sn, dateFrom...`)，呼叫 Service/DAO 層獲取數據，並以 JSON 回傳。

```
// controllers/yourController.js
exports.getData = async (req, res) => {
  const { type, sn, dateFrom, dateTo } = req.query;
  try {
    const models = await fetchDataService(type, ...);
    // 💡 Tip: 務必回傳 { models: [] } 格式以維持前端統一
    res.json({ models });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
};
```

3. 前端視圖開發 (Frontend View - EJS)

Step 3: 建立 EJS 模板

在 `views/` 目錄下建立 `.ejs` 檔案。重點包含：

1. **i18n 注入**: 將後端翻譯字串轉為前端全域變數 `i18nLabels`。
2. **DOM 元素**: 圖表容器 (`#chart`)、控制面板 (Loading, DatePicker, ProductSelect)。
3. **腳本引入**: 引入 ECharts 庫與自定義 JS。

```
<!-- views/your-module.ejs -->
<div id="chart" style="width:100%; height:600px;"></div>

<script>
  // 注入 i18n
  var i18nLabels = {
    title: "<%= __('chart_title') %>",
    speed: "<%= __('speed_label') %>",
    // ...
  };
</script>
<script src="/js/your-module.js"></script>
```

4. 前端邏輯開發 (Frontend Logic - JS)

Step 4: ECharts 配置與數據縮放 (Data Scaling)

對於不同數量級的數據 (如壓力 vs 速度)，使用 **Scaling Factor** 進行調整，使其能顯示在同一 Y 軸上，並在 Tooltip 中還原。

```
// views/js/your-module.js

// 1. 定義縮放因子
const scaleFactors = {
  pressure: 15, // 壓力放大 15 倍
  speed: 15    // 速度放大 15 倍
};

function buildChartOption(model) {
  return {
    // ...
    series: [
      {
        name: i18nLabels.pressure,
        type: 'line',
        // 2. 數據乘上因子
        data: model.pressure.map((v, i) => [i * 0.002, v * scaleFactors.pressure])
      }
    ],
  },
}
```

```
    tooltip: {
      formatter: (params) => {
        // 3. 顯示時除回因子，還原真實數值
        let val = params[0].value[1];
        if (params[0].seriesName === i18nLabels.pressure) {
          val = val / scaleFactors.pressure;
        }
        return `${params[0].seriesName}: ${val.toFixed(2)}`;
      }
    }
  };
}
```

Step 5: 自動輪詢機制 (Auto Polling)

實作 `fetchData` 與 `startCountdown` 循環。

```
let autoRefresh = true;
let timeLeft = 105;
let countdownInterval;

function fetchData() {
  clearInterval(countdownInterval);
  // 1. Show Loading
  // 2. Fetch API
  // 3. Render Chart
  // 4. Finally -> Restart Countdown
  if (autoRefresh) startCountdown();
}

function startCountdown() {
  countdownInterval = setInterval(() => {
    timeLeft--;
    updateTimerUI(timeLeft);
    if (timeLeft <= 0) {
      fetchData();
      timeLeft = 105;
    }
  }, 1000);
}
```

5. 開發檢核點 (Checklist)

1. 路由測試: 確保 `/your-module/data` 能正確回傳 JSON。
2. `i18n`: 確認所有圖表標籤都從 `i18nLabels` 讀取，無 Hard-coded 字串。
3. 縮放驗證: 檢查圖表 Y 軸是否合理，並確認 Tooltip 數值是否與原始數據一致 (已還原)。
4. 效能優化: 確保 `startCountdown` 在 `fetchData` 前被清除 (`clearInterval`)，避免重複計時器。
5. 響應式: 確認視窗縮放時呼叫 `chart.resize()`。